“植屿”系统植物识别模型训练部分

一、模块概述

1.1 功能定位

系统简介：“植屿”智慧植物养护助手系统主要为爱花养花用户打造的一款软件，用户可以首页进行拍照识别植物、查看植物的基本信息及养护方法、ai 小助手智能答疑、记录自己的养护日记（有待办提醒、拍照记录等功能）、此外也可以在植物社区进行交流及在商城购买需要的工具及绿植，形成一个闭环体验。

系统采用技术栈介绍：

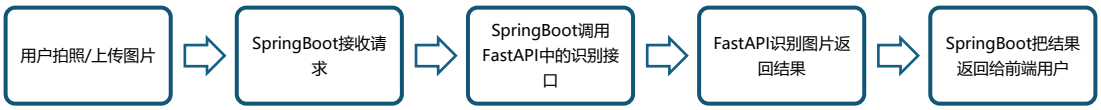
后端：SpringBoot3 + MyBatis-Plus + JWT

前端：Uni-App（用户端）、Vue3 + Element-Plus（管理员端）

数据库：MySQL + Redis

植物识别：PyTorch + MobileNetV3

植物识别功能介绍：在植物养护助手系统中植物识别功能模块是其中一个核心模块，主要实现的是用户可以对植物进行拍照或上传图片，发送到后台进行一个调用训练模型并返回结果给用户。大致流程如下：



1.2 技术路线

技术选型：该系统选用的是 PyTorch 作为深度学习的框架，选择 MobileNetV3-Large 作为基础模型。

相关技术介绍及原因：

PyTorch：PyTorch 是由 Meta 人工智能研究院开发的一个深度学习框架，目前对学术界和工业界来说是最流行的深度学习工具之一。PyTorch 本质上来说是一个用 Python 写的库，可以帮助我们搭建、训练、部署神经网络。选择 PyTorch 的原因有：PyTorch 上手快代码简洁，它的设计接近原生 Python；PyTorch 调试方便，可以打断点看变量；PyTorch 的教程和开源项目也很多，遇到问题可以查资料解决；PyTorch 对迁移学习支持很好，里面有大量预训练模型；PyTorch 部署比较方便，它支持导出 ONNX 格式，可跨平台跨语言使用；当然它也是目前学术界主流框架。

MobileNetV3：MobileNetV3 是一个轻量级卷积神经网络，是谷歌 2019 年提出的，专门为移动端和嵌入式设备设计的。选择 MobileNetV3 的原因：MobileNetV3 它轻量高效，模型文件小训练和推理都很快；再者它适合移动端，以后用户界面是跑在手机 App 上面的；其次它对迁移学习很友好，准确率够用（对我目前 40 种植物类别来说），训练速度很快（可用 GPU 训练比 CPU 快）；最后就是部署很方便了，导出 ONNX 文件后，可以用 FastAPI 加载，为 SpringBoot 使用。

二、数据集构建

2.1 类别选择

系统目前只选取了日常家庭中常养的 40 种植物，后续可能还会优化新增。我是大致分为了 8 大类，一共 40 种：观叶类绿植 13 种、草本观花类 6 种、多肉多浆 4 种、球根花卉 3 种、兰科花卉 3 种、木本观花 6 种、香草类 2 种、水生类 3 种。

植物种类	植物名称
观叶类绿植	龟背竹、橡皮树、金钱树、绿萝、虎皮兰、吊兰、文竹、散尾葵、常春藤、孔雀竹芋、海芋、合果芋、富贵竹
草本观花类	长寿花、仙客来、君子兰、白掌、蓝雪花、红掌、
多肉多浆	仙人球、芦荟、吉娃娃、玉露
球根花卉	百合、郁金香、水仙
兰科花卉	蝴蝶兰、文心兰、卡特兰
木本观花	三角梅、月季、栀子花、茉莉花、绣球花、桂花
香草类	薄荷、迷迭香
水生类	碗莲、铜钱草、睡莲

2.2 数据来源

数据来源分为两个阶段：模型训练初期、后续优化

第一阶段（模型训练初期）：主要通过 Python 脚本从网络爬取。主要来源是百度图片爬虫，因为百度在国内访问快、中文搜索准，但还是会有一些商家图片需要自己手动删除一下，这个放在数据清洗那一块；补充来源有两个，必应图片和 GBIF，从爬取的结果来看我不是很推荐使用必应爬取，必应的准确率有些低，可能是我用的免费接口的问题，GBIF 的图片质量高、标注准确，但是国内访问速度慢，我只爬取了 10 种比较冷门的植物，有一部分没有爬到需要手动下载一下了；手动下载我只要在百度和必应图片搜索用到了 Fatkun 插件，它可以提取当前窗口下载方便且速度很快。每一类差不多 150-200 张图片就好了，在后续清理结束后保留 100-150 张就好。

第二阶段（后续优化）：后续我计划去花鸟市场、小区、家等场景实拍照片进行补充，让模型更适应真实拍照场景，因为网络图片背景相对单一，实拍图可以提升模型在实际使用中的准确率。

爬取关键词示例：	
英文名	**中文搜索关键词**
monstera	龟背竹 盆栽
pothos	绿萝 盆栽
rose	月季 盆栽
cactus	仙人球 盆栽

2.3 数据清洗

数据清洗分为两部分：手动清洗+Python 脚本清洗

手动清洗：把文件夹中模糊的、水印大的、商品展示的图片手动删除一下，便于后续脚本清洗

脚本清洗：分为四个步骤

（1）统一格式：将 PNG、WEBP、BMP 等格式统一转换为 JPG 格式，保证后面训练时数据格式一致

（2）低分辨率过滤：删除分辨率低于 224×224 的图片，因为模型输入尺寸为 224×224，过小的图片放大后会模糊，影响训练结果

（3）重复图片去除：使用感知哈希算法计算每张图片的哈希值，当两张图片的汉明距离小于阈值 3 时，判定为重复图片并删除。这样子可以

去掉网络上大量转载的重复图片，避免模型过拟合

（4）统一重命名：将清洗后的图片按“类别名_编号.jpg”的格式统一重命名，如 rose_0001.jpg，方便后续数据集划分和训练时读取

清洗前后对比：原始数据共 6932 张，清洗后保留约 6582 张，平均每类 164 张。

2.4 数据划分

清洗后的数据集按照 8:1:1 的比例划分为训练集、验证集和测试集。

数据集	比例	用途
训练集	80%	用于模型参数学习
验证集	10%	训练过程中评估模型、调整超参数、选择最佳模型
测试集	10%	训练完成后评估模型最终性能，不参与训练

三、模型训练

3.1 模型选择

本次训练选择 MobileNetV3-Large 作为基础模型，它是谷歌提出的轻量级卷积神经网络，专门为移动端和嵌入式设备设计。

核心结构包括	作用
深度可分离卷积	大幅减少参数量和计算量
SE 注意力模块	自适应调整通道权重，提升特征表达能力
H-Swish 激活函数	比 ReLU 更平滑，精度更高且计算量小

选择 MobileNetV3-Large 的原因：1.轻量高效、2.适合移动端部署、3.对迁移学习友好、4.精度够用、5.部署方便

3.2 迁移学习策略

由于项目的植物数据集规模有限，从零训练容易过拟合，所以采用迁移学习策略，做法如下：

（1）加载预训练权重：使用 MobileNetV3-Large 在 ImageNet 数据集上的预训练权重作为初始参数。ImageNet 包含 1000 类、120 万张图片，模型已经学到了通用的图像特征（如边缘、纹理、形状……），这些特征对植物分类同样有效。

（2）替换分类头：MobileNetV3-Large 原始分类头输出 1000 类，所以把最后一层全连接层替换为---model.classifier[3] = nn.Linear(model.classifier[3].in_features, 40)。

（3）整体微调：训练采用整体微调策略，所有层都参与训练，因为植物图片与 ImageNet 的自然图像差异较大，整体微调的话可以使模型更好适应植物特征。

（4）优化器与学习率：使用 Adam 优化器，学习率设为 0.001，Adam 自适应调整每个参数的学习率，收敛快且稳定，适合迁移学习场景。

3.3 数据增强

担心数据可能会出现过拟合情况，训练时对图片做了一些数据增强：

增强方式	参数	作用
随机水平翻转	RandomHorizontalFlip	模拟不同拍摄方向
随机旋转	RandomRotation(15)	模拟不同拍摄角度

颜色抖动	ColorJitter(0.2, 0.2, 0.2)	模拟不同光照条件
尺寸统一	Resize((224, 224))	统一输入尺寸

注：验证集和测试集不做数据增强，只做 Resize 和归一化，保证评估结果稳定；归一化参数：mean = [0.485, 0.456, 0.406] ; std = [0.229, 0.224, 0.225]

3.4 训练参数

参数	值	说明
输入尺寸	224 × 224	与 MobileNetV3 标准输入一致
Batch Size	32	每批处理 32 张图片
Epochs	15	训练 15 轮
学习率	0.001	Adam 优化器默认学习率
优化器	Adam	自适应学习率优化器
损失函数	CrossEntropyLoss	多分类标准损失函数
训练设备	GPU（RTX 2050）	使用 CUDA 加速

3.5 训练过程

训练共进行 15 轮，每轮记录训练集和验证集的准确率，保存验证集准确率最高的模型，以下是训练关键数据：

轮次	训练集准确率	验证集准确率	备注
1	62.25%	61.56%	保存最佳模型
2	81.96%	78.12%	保存最佳模型
3	87.48%	80.00%	保存最佳模型
6	93.56%	82.50%	保存最佳模型
8	94.11%	83.75%	保存最佳模型
11	94.91%	84.69%	最佳模型
15	96.15%	82.50%	训练结束

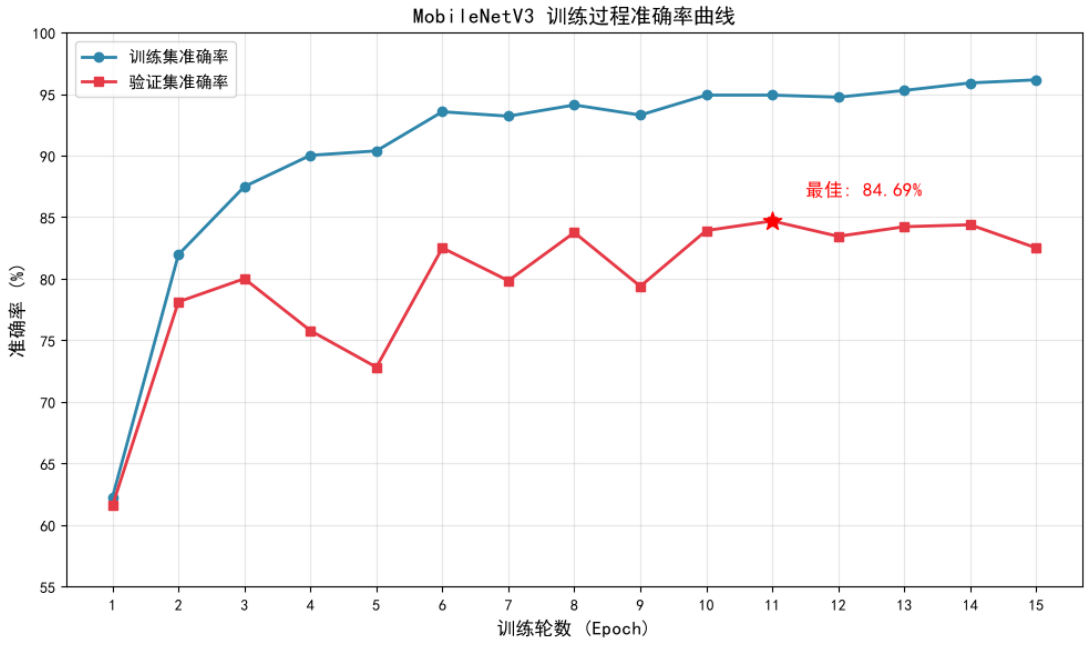
训练结论：1.模型在前三轮快速收敛，准确率从 61.56%提升到 80%；2.第 11 轮达到最佳验证准确率 84.69%，之后稍有波动；3.训练集准确率最终达到 96.15%，验证集准确率 84.69%，两者差距约 11%，属于正常范围，未出现严重过拟合；4.总训练耗时约 35 分钟

训练曲线说明：1.训练集准确率持续上升，说明模型在学习；2.验证集准确率在第 11 轮达到峰值后略有下降，说明模型开始轻微过拟合；3.选择第 11 轮的模型作为最终模型，兼顾训练集和验证集的表现

训练流程图：



训练曲线图：



四、模型评估

4.1 评估指标

模型在训练完成后需要在测试集上评估真实性能如何，采用以下指标：

指标	说明
总体准确率	测试集中预测正确的样本占总样本的比例
各类别准确率	每个植物类别单独统计的识别准确率，用于发现弱项
混淆矩阵	展示模型在哪些类别之间容易混淆

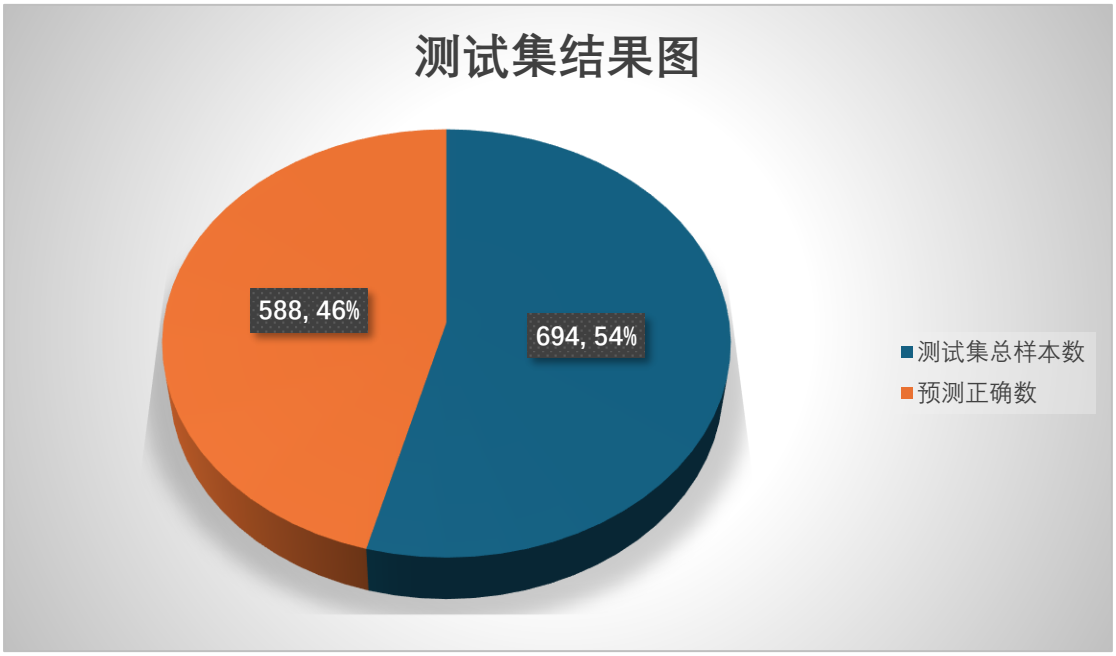
测试集说明：测试集共 694 张，40 个类别，平均每类约 17 张左右，测试集不参与训练，仅用于最后评估

4.2 测试集结果

模型在测试集上的总体准确率为 84.73%

指标	数值
测试集总样本数	694 张
预测正确数	约 588 张
总体准确率	84.73%

结论：模型在未见过的测试数据上达到了 84.73%的准确率，与验证集准确率 84.69%几乎一致，说明模型没有出现过拟合情况，泛化能力稳定



4.3 各类别分析

40 种植物的准确率结果如下：

表现优秀的植物（准确率≥90%）

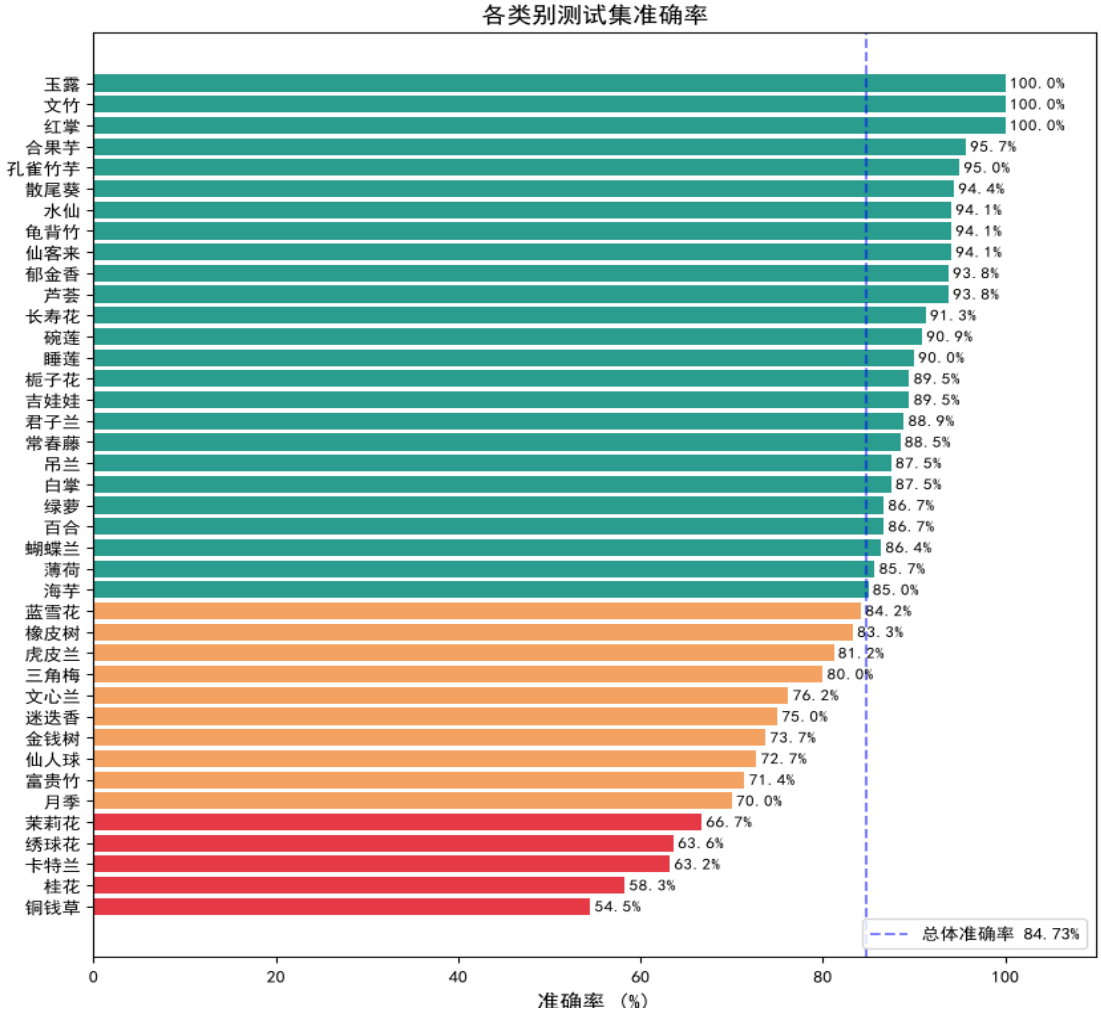
类别	准确率
红掌	100%
文竹	100%
玉露	100%
合果芋	95.7%
散尾葵	94.4%
龟背竹	94.1%
仙客来	94.1%
水仙	94.1%
郁金香	93.8%
芦荟	93.8%
长寿花	91.3%
碗莲	90.9%
睡莲	90.0%

表现较差的类别（准确率≤70%）

类别	准确率	可能原因
铜钱草	54.5%	与镜面草等圆形叶片植物混淆
桂花	58.3%	花朵小，与茉莉等白色小花混淆
卡特兰	63.2%	与蝴蝶兰、文心兰等兰科植物混淆
绣球花	63.6%	与月季、牡丹等大花植物混淆
茉莉花	66.7%	与栀子花、桂花混淆

月季	70.0%	与玫瑰、蔷薇混淆

结果分析：1.准确率高的植物类别通常是形态特征明显、数据量充足的植物，例如红掌、玉露、龟背竹……2.准确率较差的植物类别多为花朵形态相似或叶片接近的植物，模型很难仅凭视觉特征区分 3.之外，部分植物类别（如铜钱草）的训练样本相对较少也影响识别结果



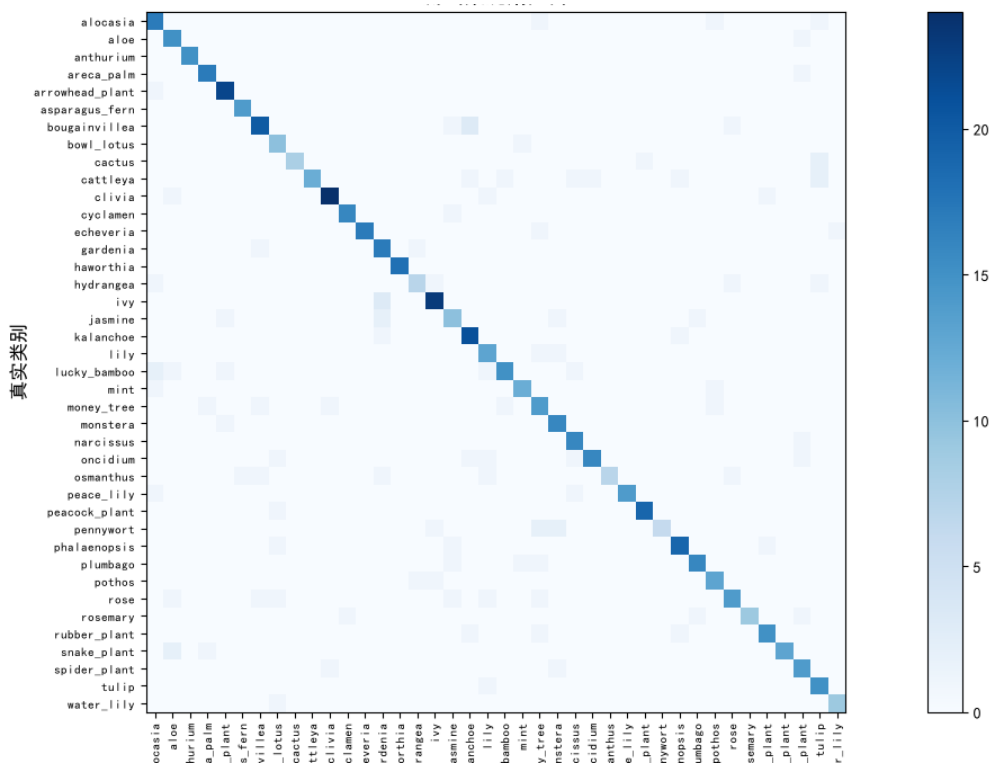
4.4 问题分析

（1）相似植物混淆：兰科植物之间容易混淆，白色小花类植物之间也容易混淆

（2）数据来源单一：当前数据来源主要来自网络爬取的图片，背景相对单一，模型对真实场景的适应能力有限

（3）部分类别样本偏少：个别冷门植物的图片数量相对较少，导致模型学习不充分

（4）改进方向：补充相似类别的差异化图片，尤其是区分特征明显的叶片、整株图；增加实拍照片，提升模型适应真实场景的能力；对样本少的类别进行数据增强或额外采集；后续可能会采用更强的模型训练或者加入注意力机制来提升准确率

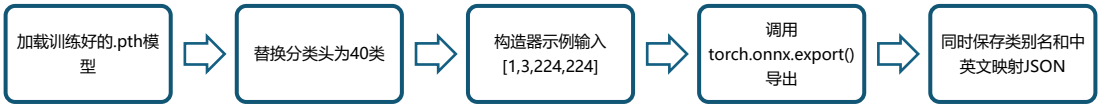


五、模型部署

5.1 ONNX 导出

为什么要导出 ONNX？→ PyTorch 训练出来的模型是.pth 格式，只能在 Python 环境下用 PyTorch 加载，但是系统的后端是使用的 SpringBoot，无法直接加载.pth 文件

ONNX 是一种跨平台、跨语言的模型格式，可以被 Java、C++、Python 等多种语言加载，所以需要把 PyTorch 模型导出为 ONNX 格式。导出流程如下所示：



导出后的文件：

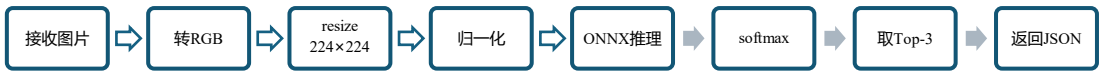
文件	作用
plant_model_xxx.onnx	ONNX 模型文件
class_names_xxx.json	英文类别名列表（顺序与模型输出一致）
class_names_cn_xxx.json	中英文映射（用于前端展示）

5.2 FastAPI 服务

SpringBoot 是 Java 生态，无法直接加载 ONNX 模型，所以需要一个 Python 推理服务，专门负责加载模型、接收图片、返回识别结果，SpringBoot 通过 HTTP 请求调用。选择 FastAPI 的原因有轻量、启动快；自带接口文档、异步支持好，适合处理图片上传；代码简洁。

功能	说明
加载 ONNX 模型	启动时加载，常驻内存
接收图片	通过 POST /recognize 接收
图片预处理	resize 到 224×224、归一化
模型推理	调用 ONNX Runtime
返回结果	返回 Top-1 和 Top-3

推理流程：



接口定义：

项目	内容
路径	POST /recognize
参数	file：图片文件（multipart/form-data）
返回	class_en、class_cn、confidence、top3

返回示例：

Body Cookie Header 4 控制台 实际请求

Pretty Raw Preview Visualize JSON utf8


```
{
    "class_en": "spider_plant",
    "class_cn": "吊兰",
    "confidence": 0.6582,
    "top3": [
        {
            "class_en": "spider_plant",
            "class_cn": "吊兰",
            "confidence": 0.6582
        },
        {
            "class_en": "alocasia",
            "class_cn": "海芋",
            "confidence": 0.1032
        },
        {
            "class_en": "peace_lily",
            "class_cn": "白掌",
            "confidence": 0.0766
        }
    ]
}
```


部署方式：python predict_api.py（服务默认运行在 http://localhost:8000）

5.3 SpringBoot 对接

对接流程：



SpringBoot 端核心组件：

组件	作用
RecognitionController	接收前端上传的图片
RecognitionService	业务接口
RecognitionServiceImpl	调用 FastAPI 并解析结果
RestTemplate	发送 HTTP 请求
RecognitionVO	返回给前端的封装对象

调用流程：

1.Controller 接收 MultipartFile

2.Service 校验文件（是否空、大小、类型）

3.用 RestTemplate 把图片转发给 FastAPI

4.FastAPI 返回 JSON

5.Service 解析 JSON，封装成 RecognitionVO

6.Controller 返回给前端

关键配置：

在 application.yml 中配置 FastAPI 地址 -->fastapi：url：http://localhost:8000/recognize

RestTemplate 设置超时 -->factory.setConnectTimeout(5000)连接超时 5 秒；factory.setReadTimeout(30000)读取超时 30 秒

异常处理：

异常	处理
FastAPI 服务不可达	返回“识别服务暂时不可用”
图片格式错误	返回“图片格式不正确”
其他异常	返回“识别失败，请稍后重试”

5.4 接口说明

接口路径：POST /recognition/recognize

请求参数：

参数	类型	必填	说明
file	MultipartFile	是	植物图片，≤10MB，格式为 jpg/png/webp

返回示例：

字段	类型	说明
code	Integer	状态码（如 200 成功）
msg	String	提示信息
data.classEn	String	英文类名
data.classCn	String	中文名
data.confidence	Double	置信度

data.top3	Array	Top-3 候选结果

接口测试：使用 Postman 或者 Apifox 进行接口测试

1.请求方式选 POST

2.URL 填 http://localhost:8080/recognition/recognize

3.Body 选 form-data

4.key 填 file，value 选一张图片

5.发送，查看返回结果

六、总结与展望

6.1 已完成工作

本文档围绕“植屿”智慧植物养护助手系统的核心功能—植物识别，完成了从数据采集到模型部署的全流程工作，具体内容如下：

（1）数据集构建

确定 40 种日常家庭常见植物作为识别对象，分为观叶绿植、草本观花、多肉多浆、球根花卉、兰科花卉、木本观花、香草类、水生类 8 大类；通过百度爬虫、GBIF API、Fatkun 插件手动下载等多种方式采集图片；原始数据共 6932 张，经过手动清洗和 Python 脚本清洗后保留 6582 张，平均每类 164 张；按 8:1:1 比例划分训练集、验证集和测试集

（2）模型训练

选择 MobileNetV3 作为基础模型，使用 ImageNet 预训练权重进行迁移学习；采用数据增强策略防止过拟合；训练参数-->Batch Size = 32、Epochs = 15、学习率 = 0.001、优化器 Adam；最佳验证集准确率达到 84.69%，训练过程稳定，未出现严重过拟合

（3）模型评估

在测试集上评估，总体准确率为 84.73%，与验证集结果一致，说明模型泛化能力稳定；40 个类别中，红掌、玉露、文竹等类别准确率达到 100%，铜钱草、桂花、卡特兰、绣球花等类别准确率偏低；准确率偏低的主要原因可能是相似植物混淆，数据来源单一，部分类别样本偏少

（4）模型部署

将 PyTorch 模型导出为 ONNX 格式，实现跨语言调用；使用 FastAPI 搭建推理服务，加载 ONNX 模型，提供/recognize 接口，返回结果；SpringBoot 通过 RestTemplate 调用 FastAPI，完成系统与模型的对接，并设计文件校验、超时处理、异常分类等机制；对外提供统一的 POST /recognition/recognize 接口，支持用户端上传图片并获取识别结果

6.2 不足与改进

虽然植物识别模块完成基本功能，但是仍然存在不足，后续还需要不断优化

（1）数据层面

不足	改进方向
网络图片背景单一	后续补充花鸟市场、家庭等真实场景实拍图

部分冷门植物样本偏少	针对性采集，保证每类至少 150 张
相似植物混淆严重	补充能体现区分特征的叶片，整株图

（2）模型层面

不足	改进方向
40 类准确率约 84.7%，尚未达到 90%及以上	增加训练轮数，加入学习率衰减策略
相似类别混淆	尝试更强的模型或者加入注意力机制
模型未做量化压缩	后续可以尝试 INT8 量化，减小模型体积，提升推理速度

（3）系统层面

不足	改进方向
FastAPI 与 SpringBoot 为分离部署	后续可以用 ONNX Runtime for Java 直接在 SpringBoot 中推理，减少服务依赖
识别结果未入库	增加识别记录表，支持历史查询和数据分析
未做置信度兜底	当置信度低于阈值时提醒“可能不在识别库中”

（4）功能扩展

拓展功能	说明
识别库扩展	目前只覆盖 40 种植物，后续可扩展到 100 种以上，覆盖更多家庭园艺场景
病虫害识别	在现有模型基础上增加叶片病害识别功能，辅助用户判断植物健康状态
多模态识别	结合文本描述和图片，提升识别与诊断的准确性
个性化推荐	根据用户识别记录和养护习惯，推荐适合的植物和养护方案

6.3 总结

本模块完成了“植屿”系统植物识别功能的完整技术闭环：从数据采集与清洗、到模型训练与评估、再到 ONNX 导出与 FastAPI 部署，最终由 SpringBoot 对接供前端调用，整个流程覆盖了深度学习项目从数据到落地的各个环节，模型在测试集上达到 84.73%的准确率，满足系统要求。

后续会通过补充实拍数据、优化模型结构、扩展识别类别，识别准确率和实用性还能进一步提升，为系统的其他功能模块（如养护日记、AI 小助手、植物社区）提供可靠的技术支撑。